

Claude Chat, Code & Cowork — Hands-on Lab

Hexagon · Innovation Hub · 22 September 2026

A step-by-step lab for working with Claude across Chat, Code, and Cowork. You will take one real bug — **DASH-147**, a sales-dashboard date filter that does not actually filter the chart — from first diagnosis all the way to a closed ticket, using your own Claude account throughout.

You already have everything you need. You will work in your own Claude — the same Claude Cowork and Claude Code seats your team already uses. There is nothing to buy and no AWS account to set up. Ignore any mention elsewhere of AWS Marketplace, event invites, or buying a new organization; none of that applies here.

How to get the most from this lab

The point of the next 90 minutes is **not** to type the prompts and watch them work. DASH-147 is a stand-in for your own backlog — a bug nobody fully understands, in code nobody has touched in months. As you go, keep asking *"what is Claude actually doing here, and where would I use this on my own work?"*

To help with that, every lab has two framing aids:

- **What you'll learn** — the transferable skill the lab is really teaching, stated up front. The DASH-147 steps are just the vehicle.
- **In real life** — short notes that map the exact step you just did onto the kind of work you do day to day.

One idea runs under all of it: Claude works best when it is **grounded in real context** and **given a way to check its own work**. Watch for those two threads — they are what separate a session you can trust from one you cannot.

The scenario — what we're simulating

For the next 90 minutes you are a developer on the team that owns a customer-facing **sales dashboard**. Picture the situation many of you know well: the dashboard was built at pace, the person who wrote it has moved on, and the tests are thin.

This morning a product manager, Sarah, files a bug — **DASH-147**:

"When I pick 'Last 7 Days' on the dashboard's date filter, the chart still shows all-time data. The number looks wrong in every review. Can someone take a look?"

You have never opened this code before. Over the lab you will play out the **full life of that ticket** the way it really happens at work — except with Claude alongside you at each stage:

- 1 **Understand it** — work out what is actually broken, in code you did not write (Chat + Projects, Lab 1).
- 2 **Fix it** — make the change safely across the files that need it (Code mode, Lab 2).
- 3 **Get it reviewed** — a second opinion before it ships (/code-review, Lab 3).

- 4 **Show it** — prove the fix to the PM who filed it, without making her read code (Artifacts, Lab 4).
- 5 **Keep it safe** — a teammate's new feature landed on top of yours; catch what it introduced (/security-review, Lab 5).
- 6 **Close it out** — write the handoff and update the ticket so the next person is not guessing (Cowork, Lab 6).

Nothing here is Claude-specific make-believe: it is the everyday loop of a *bug report comes in, a closed ticket goes out*. DASH-147 is deliberately small so the mechanics stay visible — but every step maps onto the messy, real work you will point Claude at tomorrow. That mapping is what the **In real life** notes throughout are for.

What you will do

Lab	What you'll practise	Surface	Time
0	Set up & sign in	Web + Desktop	~5 min
1	Grounding Claude in real context, and diagnosing before fixing	Web	~18 min
2	Letting Claude change real code safely, from a reviewed plan	Desktop	~22 min
3	Getting a second, reasoned opinion before you ship	Desktop	~15 min
4	Turning a fix into something a non-engineer can see	Web	~12 min
5	Catching the flaws your tools and your eyes miss	Desktop	~13 min
6	Handing over a goal, not a script, and closing the loop	Desktop	~10 min

Lab 0 — Set up & sign in

~5 min · **your own Claude account and a modern browser**. The sign-in and install steps are optional if you are already set up.

Already using Claude? Most of this lab is optional. Since your team already has access — and most of you already have Claude on the web and the desktop app installed and signed in — only Step 1 (getting the sample files) is required. If you are already signed in on both web and desktop, grab the files and jump to Lab 1.

Step 1 · Get the sample files (*required*)

The whole lab runs against one small sample project — **DASH-147** — four files:

- `bug-report.md` — the ticket, as a PM filed it

- `DateFilter.jsx` — the dropdown component
- `Dashboard.jsx` — the parent component
- `ChartPanel.jsx` — the chart component

- 1 Download the dash-147 bundle: <https://raw.githubusercontent.com/main/dash-147.zip>
- 2 Unzip it into a local folder you can find easily (for example your Desktop). Keep the unzipped folder — you upload the files in Lab 1 and point Code mode at this folder in Lab 2.

Watch out — upload the files, not the zip, and don't delete the hidden `.git` folder. The dash-147 folder is already a git repository, with the original buggy code committed as a baseline. That is what lets the review lab compare your fix against the original. You never touch git yourself, but if you delete the hidden `.git` folder, Lab 3 will not work.

Step 2 · Sign in to Claude on the web (*optional — skip if already signed in*)

- 1 Go to <https://claude.ai> and sign in with your usual work account — the same one you already use for Claude at Hexagon.
- 2 Claude may send a magic link or a verification code to your email. Open the email and click the link, or enter the code.
- 3 Confirm you land in your workspace. The left sidebar shows **Chats** and **Projects**, with your organisation name near the bottom.

Watch out — no magic-link email? Check your spam/junk folder first — it is the single most common hold-up. If nothing arrives after a minute, click **Try sending again** on the verification screen. Typed the wrong address? Click **Change email address** and start over.

Optional connection check: paste `what is today's date?` into the web chat. A reply with the current date means your web session is live.

Step 3 · Install and sign in to Claude Desktop (*optional — skip if already installed*)

The desktop app is what gives you Code and Cowork against a real folder on your machine, and you will need it from Lab 2 onward. **Skip this if you already have Claude Desktop installed and signed in** — most of you will.

- 1 Download Claude Desktop from <https://claude.com/download> and install it. On macOS, when it asks whether to open an app downloaded from the internet, click **Open**.
- 2 Launch Claude from your Applications folder (macOS) or Start menu (Windows).
- 3 Sign in to the same account. Enter your work email; Claude sends a code or a magic link.
- 4 Confirm you land on the Chat home, with your organisation name at the bottom of the sidebar.

Watch out — magic link instead of a code? If Claude emails a magic link rather than a code, click **Sign in to Claude.ai** in the email. Your browser opens a page asking whether to open the Claude app — click **Open Claude** to finish. If the browser does not hand off to the app, make sure the desktop app is already running, then click the link again.

Optional connection check: paste `what is today's date?` into the desktop chat.

Before you move on

- You have the four dash-147 sample files in a local folder (with the hidden `.git` folder intact).
- You can reach Claude on the web (already signed in, or via Step 2).
- Claude Desktop is installed and signed in to the same account (already, or via Step 3) — needed from Lab 2 on.

Lab 1 — Diagnose DASH-147 with Chat

~18 min · Web app · Prerequisites: signed in to Claude on the web; the four sample files downloaded.

Goal. Give the team a shared, grounded place to reason about the dashboard — a Claude **Project** — then use Chat to trace the bug to its root cause. You will not change any code here; the output is a correct diagnosis you act on in Lab 2.

What you'll learn

- How to **ground Claude in your own material** so its answers come from your files, not from generic knowledge of how React "usually" works.
- Why a **Project** is the team's shared starting point — one set of instructions and files that every conversation inherits, so nobody re-explains the context.
- The discipline of **diagnosing before fixing**: getting the root cause right while it is cheap, before any code changes.

Why a Project?

A **Project** bundles custom instructions and uploaded files so every conversation inside it starts grounded in the same materials. You create it once here and reuse it across diagnosis, fixing, and prototyping. Anthropic describes Projects as a way to ground Claude's output in your internal knowledge — your codebase, style guides, past work — instead of re-pasting it into every new chat.

In real life. This is the fix for "I explain the same background to Claude every single time." A Project is where you would put your team's coding conventions, a service's key docs, a design system, or the reference files for a recurring report — so every teammate's chat starts from the same trusted context and the answers stay consistent across the team.

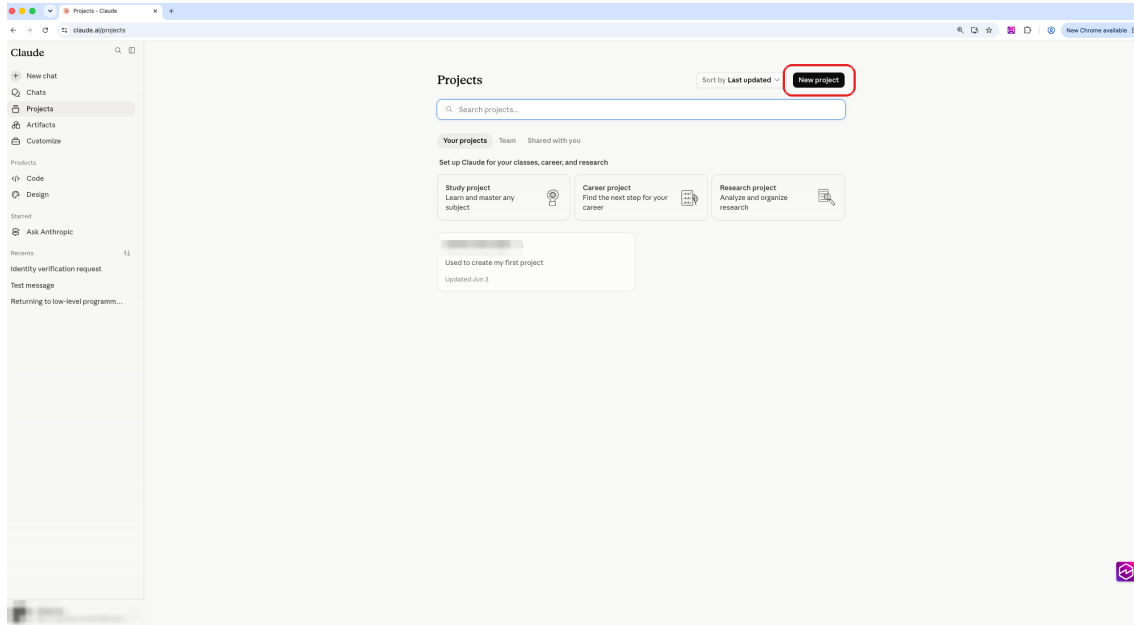
Step 1 · Create a grounded Project

- 1 In the left sidebar, click **Projects**, then **New project** in the top-right.
- 2 Name it Frontend Dashboard, add a short description, and create it.
- 3 Open the instructions dialog (the edit control on the **Instructions** panel) and paste the framing below, then save:

You are helping diagnose and fix bugs in a React dashboard application. The application uses React 18 with functional components and hooks. Component hierarchy: Dashboard (parent) -> DateFilter + ChartPanel (children). When analyzing code, trace the data flow from user

interaction to rendered output. Ground every answer only in the uploaded project files; if something is not in the files, say so.

- 1 On the **Files** panel, add all four sample files: `bug-report.md`, `DateFilter.jsx`, `Dashboard.jsx`, `ChartPanel.jsx`.
- 2 Confirm all four appear in the Files panel on the right.



The claude.ai Projects page with the New project button highlighted in the top-right

Why that last line matters. The instruction "ground every answer only in the uploaded files; if something is not in the files, say so" is what stops Claude giving a generic React answer. It also gives it permission to say "that isn't in the files" instead of inventing something — which is exactly what you want when the stakes are real. It is the single most useful sentence in the whole Project.

Step 2 · Trace the bug to its root cause

Start a new chat *inside the Project* so the conversation draws on the uploaded files, and send:

Using only the uploaded project files, analyze the data flow from when a user selects "Last 7 Days" in `DateFilter` through to what `ChartPanel` finally renders. Where does the flow break down, and what is the root cause? Cite the specific files you draw from.

Read Claude's diagnosis and confirm it names the real root cause: **`DateFilter` holds the selected range in its own local state and never notifies `Dashboard`**, so `Dashboard` keeps passing unfiltered, all-time data to `ChartPanel`.

In real life. Asking Claude to *trace the flow and name the root cause, citing the files* is how you onboard onto code nobody owns any more, or investigate a bug you did not write. Anthropic's own guidance for a new codebase is to ask the questions you would ask a senior engineer — "how does this work, where does it break, why is it built this way" — before you touch anything. The habit that matters: make Claude show you the cause and its evidence before it proposes a fix.

What a good answer looks like. Claude names the four files, traces the dropdown selection getting trapped in DateFilter's local state, and explains why the chart always shows all-time data — grounded in your code, not a textbook answer. If it stays generic, remind it: "Answer only from the uploaded files."

Optional — explore what Chat can do

- Ask for a plain-English summary of DASH-147 for a non-technical stakeholder, then a one-line Slack version.
- Have Claude propose two or three fix strategies for the state-lifting issue and lay out the trade-offs — without writing code yet.
- Ask it to draft a regression-test checklist a QA engineer should run once the bug is fixed.
- Turn on extended thinking for a follow-up and compare the response.

Before you move on

- You created a Project named *Frontend Dashboard* with saved instructions.
- All four sample files appear as Project knowledge.
- You ran the diagnosis prompt inside the Project.
- Claude's answer cites the uploaded files and names the state-lifting root cause.

Lab 2 — Fix DASH-147 with Code mode

~22 min · Claude Desktop · Prerequisites: Claude Desktop signed in; the diagnosis from Lab 1; the dash-147 folder.

Goal. Switch from reasoning over uploaded copies (Chat) to editing the real files (Code). You point Code mode at your folder and land the coordinated, multi-file fix — reviewing the diffs before they touch disk.

What you'll learn

- The difference between Claude **advising** you (Chat) and Claude **acting** on your actual files (Code) — and why acting-on-real-files is what makes a coordinated, multi-file change possible.
- How to work the **explore plan implement verify** loop that Anthropic recommends: understand first, agree the change, then let Claude make it — and check its own work — rather than letting it jump straight to code.
- How to **stay in control** by reviewing the proposed diff *before* it touches disk, instead of reviewing code after the fact.

Code mode vs Chat

Chat reasons over uploaded copies of your files. **Code mode connects to a local folder** with full read and edit access, proposes diffs you approve, and applies them to disk. That is what makes a coordinated change across several files possible — the kind DASH-147 needs. The mental shift Anthropic calls out: instead of writing the code yourself and asking Claude to review it, you describe the outcome and Claude explores, plans, and implements while you supervise.

Step 1 · Connect Code mode to your codebase (explore first)

- 1 In Claude Desktop, open **Code** (the Code tab / mode selector).
- 2 Choose **Open folder**, navigate to your dash-147 folder, and open it.
- 3 **Trust the workspace.** Claude asks if you trust this folder — the path shows at the bottom of the dialog. Confirm it is the right folder, then click **Trust**.
- 4 Get a grounded tour before changing anything:

Give me a tour of this codebase, then trace the data flow from the date filter to the chart: which component owns the selected range, and how does that selection reach the chart that renders the data?

Confirm the answer names the four files and restates the Lab 1 root cause — `DateFilter` owns the selected range in local state, and the selection never reaches `Dashboard` or `ChartPanel`. That tells you Claude is reading your real files.

In real life. The "give me a tour, then trace one flow" opening is the fastest way to get productive in a repository you do not know — a service you have inherited, a teammate's project, an open-source dependency you need to patch. You are confirming Claude is grounded in the *real* files before you let it change anything, which is the whole point of exploring before implementing.

Step 2 · Land the multi-file fix (plan, then implement)

Describe the fix as one concrete, multi-part change — and ask to see the diffs first:

Fix DASH-147. `DateFilter` holds its selected range in local state but never passes it up. Add an `onFilterChange` callback to `DateFilter`, lift the filter state into `Dashboard`, filter the data by the selected range before passing it to `ChartPanel`, and add boundary logic for "Last 7 Days", "Last 30 Days", and "All Time". Show me the diffs before applying them.

Review the proposed diffs, then tell Claude to apply them:

Yes, apply these edits.

- 1 **Read the summary.** Claude writes the changes and reports one line per edited file with its added/removed line counts.
- 2 **Expand a diff.** Click the chevron next to a file to read the exact changes.
- 3 **Notice what changed — and what didn't.** Only `DateFilter.jsx` and `Dashboard.jsx` change. `ChartPanel.jsx` is left untouched: it already renders whatever data it is handed.

In real life. "Show me the diffs before applying them" is the single habit that makes agentic coding safe to adopt. Reading a wrong *plan* costs a minute; reading a wrong *implementation* after it has landed costs an afternoon. Notice the shape of a good request, too: it states the root cause, lists the concrete changes, and points at the specific components — the more precise the brief, the fewer corrections you make. That is exactly how you would write it against your own repo.

Why `ChartPanel` needed no edit. The real fix was moving *ownership* of the selected range up to `Dashboard` — not editing every file. The range value itself never reaches `ChartPanel`; only its consequence, the already-filtered data, does. Lifting state up is the pattern, and Claude applied it precisely — a reminder that the right change is often smaller and more surgical than "touch everything that looks related."

Step 3 · Verify the fix end to end

Walk me through the corrected data flow from the moment a user picks "Last 7 Days" to what the chart renders, and confirm the selected range now reaches `ChartPanel`.

Claude should trace the full round-trip: `DateFilter` forwards the value up, `Dashboard` takes ownership and derives the 7-day slice, and that filtered data flows into `ChartPanel`. Asking for the reasoning both verifies the change and teaches the pattern.

In real life. Having Claude walk the corrected flow back to you is a lightweight verification step — you are making it show its work, not just assert "done." On your own code the stronger version is to give it a check it can run itself: "write a failing test first, then fix it, then run the suite." A change Claude can prove is right is one you can accept without re-reading every line.

Optional — go further

- Ask Claude to explain the new boundary logic in plain English and what "Last 7 Days" returns.
- Have it generate a clear git commit message for the multi-file fix.
- Ask it to add a loading state to `ChartPanel` while filtered data recomputes, and review the diff.
- Point Code mode at a repo of your own and ask for a grounded tour of one feature — no edits.
- Ask it to find other components with the same state-lifting anti-pattern.

Before you move on

- Code mode is connected to the dash-147 folder and you trusted the workspace.
- Claude's tour named the four files and restated the root cause.
- You applied the fix and expanded the diffs to confirm only `DateFilter.jsx` and `Dashboard.jsx` changed.
- Claude can trace the corrected flow from the dropdown to the filtered chart.

Lab 3 — Review the fix with `/code-review`

~15 min · Claude Desktop · Prerequisites: the DASH-147 fix applied in Code mode (Lab 2).

Goal. "Works" is not "reviewed." You will commit the fix, run Claude's built-in `/code-review` on it, read how it validates the change and surfaces one optional improvement, apply that, and re-run the review to confirm it comes back clean.

What you'll learn

- Why a **fresh-context review** is worth running: a reviewer that sees only the diff — not the reasoning that produced it — judges the change on its own terms, the way a second engineer would.
- How to read a review as a **signal you can act on**: a clean pass with reasons is a real result, not a non-event.
- The habit of switching to **ask-permissions mode** when you want to see and approve each action, versus letting Claude run freely.

How `/code-review` works

`/code-review` inspects a diff in your local git repository. The `dash-147` folder is a git repo — the original buggy code is the first commit — so once you commit your fix, the review reviews it as the change against the previous commit. It runs in a *fresh* context: because the reviewer did not write the code, it is not biased toward it — the same reason you would ask a colleague, not yourself, for the final look.

Step 1 · Switch to ask-permissions mode, commit, and review

- 1 Confirm you are still in Code mode pointed at the folder you fixed in Lab 2.
- 2 Switch the permission mode to **ask** using the control at the bottom-left of the window. In Lab 2 you let Claude apply edits directly; here you switch to the stricter mode so you can see and approve each action it takes.
- 3 Commit the fix so the review has a clean diff to inspect:

```
Commit the DASH-147 fix with a clear message describing the change.
```

- 1 Run the review — type the command in the message box:

```
/code-review
```

- 1 **Approve each git command.** Because you are in ask-permissions mode, Claude asks before running each command it needs to gather the diff — click **Allow** to let it proceed.
- 2 **Read the result.** The review confirms the fix is correct — state lifted properly, date-boundary math right, timezone handling consistent — and surfaces one optional, non-blocking improvement: the filtered data in `Dashboard.jsx` recomputes on every render and could be wrapped in `useMemo`.

In real life. `/code-review` is the second pass that lets you ship with confidence — run it on your own change before you open the PR, so you find the gaps before a reviewer (or production) does. Permission modes are the other real-world lesson here: use the safety-checked or ask mode on unfamiliar or sensitive work where you want to see every action, and save the faster "just do it" mode for throwaway branches you can discard.

A clean review is a real result. `/code-review` is not only there to find bugs. Here it validates that your Lab 2 fix is correct and at the right depth, then points out one purely stylistic improvement. Knowing your change is sound — *with reasons* — is exactly what a review is for.

Step 2 · Apply the improvement and confirm a clean pass

Apply the `useMemo` cleanup the review suggested for `filteredData` in `Dashboard.jsx`.

Claude proposes the edit and asks permission to write it. Review the change and click **Allow**. Then commit it:

```
Commit this cleanup with a clear message.
```

Re-run the review to confirm the change is still sound:

```
/code-review
```

Confirm the second pass comes back clean — "No bugs found," with the memoization verified as correct and idiomatic.

In real life. One caution worth carrying back to work: a reviewer asked to find gaps will usually find *something*, even when the code is sound. Chasing every suggestion leads to over-engineering. Take the improvements that affect correctness or your stated requirements, and treat the rest as optional — exactly as you did with the useMemo note here.

Optional — explore the review commands

- Run `/security-review` on the same folder and read how its focus differs (you will use it properly in Lab 5).
- Ask Claude to write a unit test that pins the "Last 7 Days" boundary to exactly seven points, then run it.
- Point `/code-review` at a change in one of your own repos and see what it flags.
- Ask Claude to summarise the review as a short PR description a reviewer could read in under a minute.

Before you move on

- You switched to ask-permissions mode, committed the fix, and ran `/code-review` on it.
- The review confirmed the fix is correct and surfaced the optional improvement.
- You applied the improvement, approved the diff, and committed it.
- A second pass came back clean.

Lab 4 — Prototype the fixed widget as an Artifact

~12 min · Web app · Prerequisites: signed in on the web; the DASH-147 context from Labs 1–3.

Goal. Show the fixed behaviour to someone who will never open the code. You build an interactive **Artifact** of the date-filtered chart, iterate it live, and share it as a link a product manager can click through.

What you'll learn

- How to turn a result into something a **non-engineer can see and click** — not a screenshot or a description, but a working prototype.
- How to **iterate in place** through conversation, refining the same Artifact instead of starting over.
- Where this fits: **fast sign-off and shared understanding**, without leaving Claude or waiting on a build.

What an Artifact is

An **Artifact** is a live, interactive preview Claude renders beside the conversation. It runs real React with a fixed set of libraries, so a prototype behaves like the real widget, not a static mockup. You refine it by sending follow-up messages in the same conversation, and share the finished result as a link.

Step 1 · Generate version 1 and confirm the filter works

- 1 Open the Claude web app and start a new conversation *in the Frontend Dashboard Project* from Lab 1, so the bug report and components are available.
- 2 Describe the widget you want as an Artifact:

Build an interactive Artifact of the fixed sales dashboard widget: a line chart with a date-range dropdown (Last 7 Days, Last 30 Days, All Time). Selecting a range must actually filter the chart data, not just relabel it. Use sample data.

- 1 Wait for the Artifact to render in the preview pane beside the conversation.
- 2 Click through the dropdown in the live preview and confirm the chart re-renders for each range — this is the fixed behaviour from Lab 2, now visible without reading any code.

In real life. Artifacts are how you get a decision unstuck. Instead of describing a change in a document and waiting for a meeting, you hand the stakeholder a working thing they can click. The same move builds an interactive calculator for a proposal, a data explorer for a report, or a clickable mock of a workflow — anything that lands faster when people can try it rather than read about it.

Step 2 · Iterate and share

Refine the prototype with a follow-up. Claude updates the *same* Artifact in place rather than starting over:

Add a summary stat above the chart showing the total for the selected range, and restyle the chart with clearer axis labels and a legend.

- 1 Verify the new pass in the live preview — the summary stat should update as you change the range.
- 2 Share it. Use the share control on the Artifact and copy the generated link a PM can open in a browser.

In real life. Refining by conversation — "now add a summary stat, now make it clearer" — is the natural way to work when you cannot picture the finished thing up front. You steer toward it one message at a time, and the shareable link means the person who filed the request can see the result without you scheduling anything.

Optional — push the Artifact further

- Add a second chart type (bar or area) and a toggle to switch views.
- Add a loading state and an empty state for when a range returns no data.
- Make the widget responsive so it looks right on a narrow screen.
- Ask Claude to export the Artifact's component code so a developer could drop it into the real dashboard.

Before you move on

- Claude generated an interactive Artifact with a working date-range dropdown.
- Selecting a range actually filters the chart data in the live preview.
- You iterated the Artifact at least once and verified the new pass.
- You shared the Artifact as a link.

Lab 5 — Find and fix flaws with `/security-review`

~13 min · Claude Desktop · Prerequisites: Claude Desktop signed in; the fix and review from Labs 2–3.

Goal. A teammate just added a new "chart annotations" feature on top of your work, and it has not been security-reviewed. You point Claude Desktop at that updated codebase, run `/security-review`, read the real vulnerabilities it finds, and fix them before the feature ships.

What you'll learn

- That some flaws are **invisible to your normal tools and your eyes** — a framework's safety net does not catch a developer who deliberately steps around it, and a leaked secret looks like ordinary code.
- How to run a **focused security pass on a change** and read its findings, then fix the *root cause* rather than paper over the symptom.
- Why you **re-run the check** to confirm a fix actually closed the issue, instead of assuming it did.

Why a fresh codebase

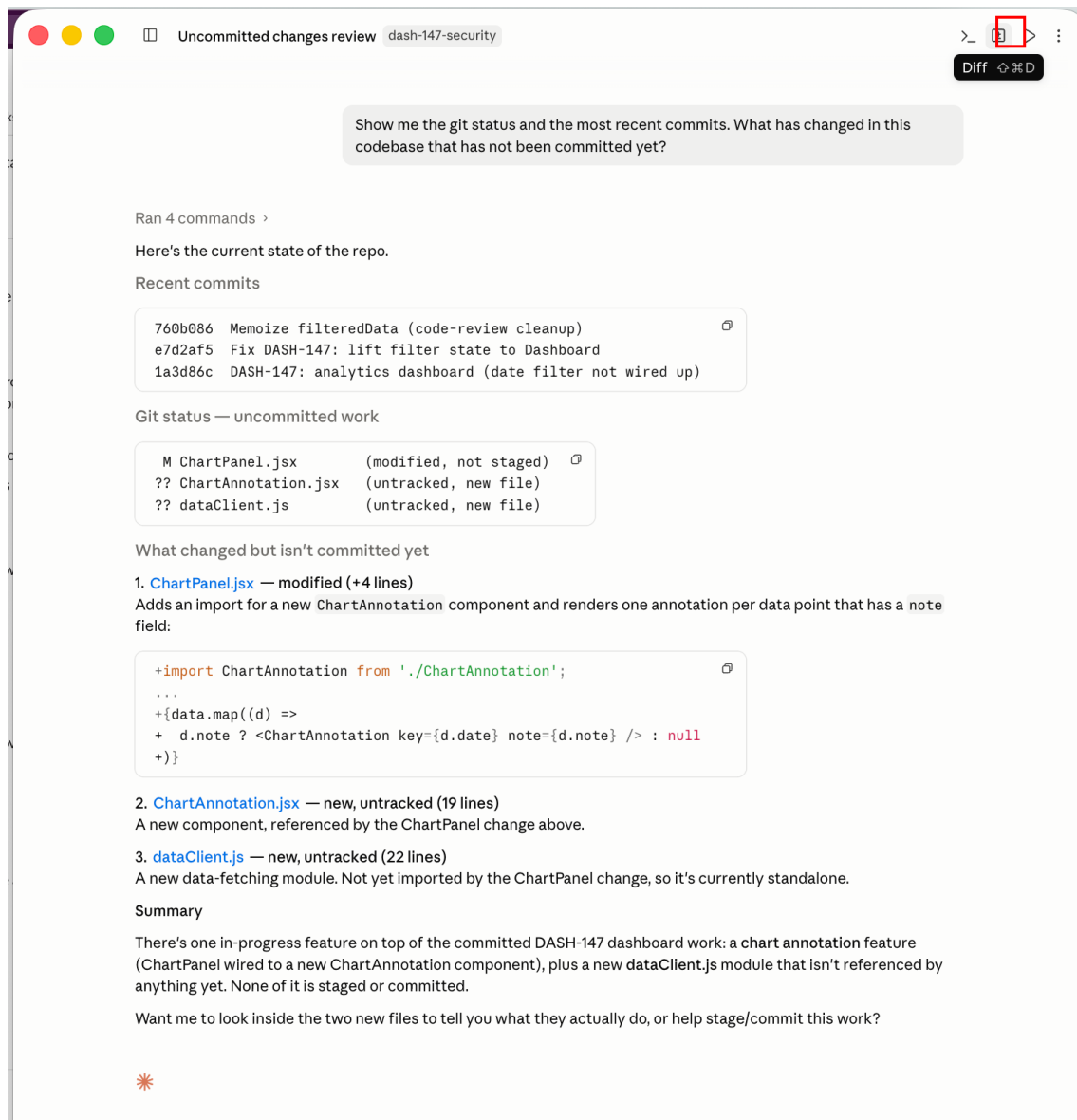
The dash-147-security bundle picks up where Labs 2–3 left off: your fix and the useMemo cleanup are committed, and on top a teammate added a chart-annotations feature — a new `ChartAnnotation.jsx` and a `dataClient.js` that loads analyst notes from an API. That feature ships as *uncommitted* changes, which is exactly what `/security-review` inspects.

The vulnerabilities are planted on purpose. They are the kind of issue React would not catch for you: a deliberate escape from React's built-in protection, and a secret committed to source. This is a clean, first-encounter review.

Step 1 · Get the updated codebase

Download the dash-147-security bundle — <https://raw.githubusercontent.com/147/h-147-security.zip> — unzip it, open it in Claude Desktop's Code mode (trust the workspace), then confirm what is pending:

```
Show me the git status and the most recent commits. What has changed in this codebase that has not been committed yet?
```

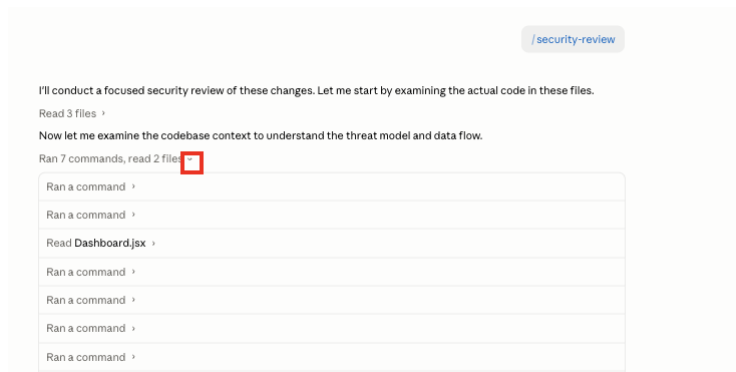


Claude Desktop showing recent commits and git status with `ChartPanel.jsx` modified plus two untracked new files

In real life. Starting a review by asking "what has actually changed here that isn't committed yet?" is how you scope any review to the work in front of you — the pending change on your branch, not the whole history. It is the same first question whether you are reviewing your own work or a teammate's pull request.

Step 2 · Run the security review

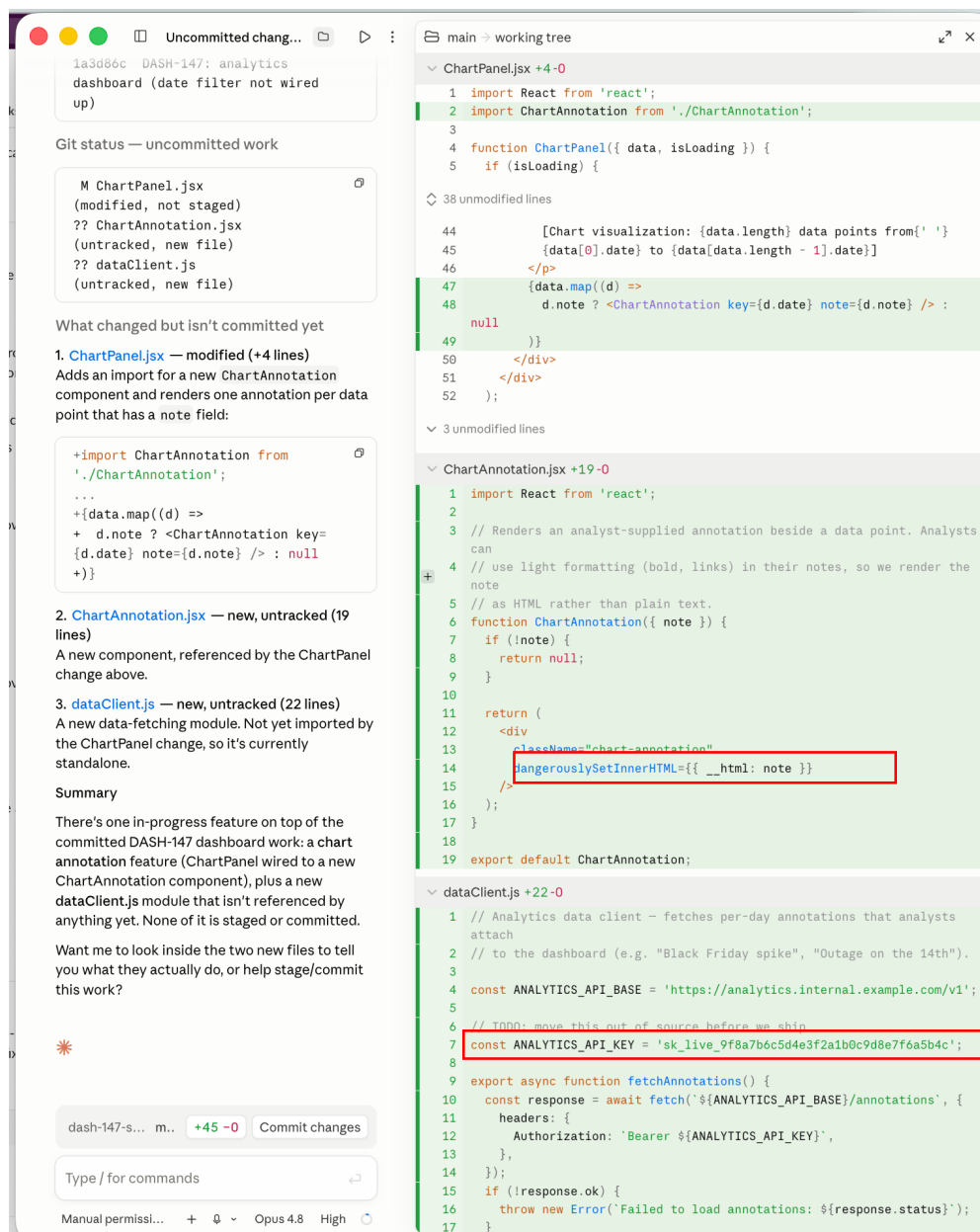
/security-review



The /security-review command running, reading files and running several commands

- 1 Approve each git command** (you are in ask-permissions mode) so Claude can gather the diff.
- 2 Read the findings.** The review should report both planted vulnerabilities:

File	Finding	What it means
ChartAnnotation.jsx	Cross-site scripting (XSS)	An analyst note is rendered with dangerous <code>lySetInnerHTML</code> , so malicious HTML or script in a note runs in the browser.
dataClient.js	Hardcoded secret	A live API key is committed directly in source, exposing it to anyone with repository access.



The diff view highlighting `dangerouslySetInnerHTML` in `ChartAnnotation.jsx` and a hardcoded `ANALYTICS_API_KEY` in `dataClient.js`

Watch out — if your first pass looks thin, run it again. /security-review fans out to its own subagents and applies a confidence threshold before reporting, so the same code can come back with both findings on one pass and only one, or none, on the next. That is the command being cautious, not a broken lab. If the first pass reports fewer than two, run /security-review again — both vulnerabilities are present.

In real life. These are exactly the flaws that slip through: the framework escapes untrusted input for you by default, so the only way an injection hole exists is if someone deliberately bypassed that protection — and a hardcoded credential reads like any other line of code. A security pass on every change is how you keep both out of production. On your own repos, the leaked-key finding also means rotating the key: once a secret is in git history, removing it from the file is not enough.

Step 3 · Fix both findings and confirm a clean review

Fix both security findings. For the XSS, render the annotation as plain text instead of using `dangerouslySetInnerHTML`, or sanitize it. For the hardcoded key, move it out of source and read it from an environment variable. Show me the diffs before applying them.

Review the diffs, approve them, then re-run the review:

```
/security-review
```

Confirm the second pass no longer reports the XSS or the hardcoded secret.

In real life. Re-running the review after the fix is the part people skip — and it is the part that proves the issue is actually closed rather than assumed closed. "Fix it and show me the check now passes" is a habit worth carrying to every change that matters.

Optional — go deeper

- Ask Claude for a short, plain-English exploit for the XSS — what a malicious annotation would contain — so you can explain the risk to a non-engineer.
- Have it explain the difference between escaping and sanitizing untrusted HTML, and when to use each.
- Ask how the leaked API key should be rotated and what to check in git history now that it was committed.
- Run `/security-review` on a change in one of your own repos.

Before you move on

- You opened the `dash-147-security` bundle and confirmed the annotations feature is uncommitted on top of the committed fix.
- `/security-review` reported the XSS in `ChartAnnotation.jsx` and the hardcoded key in `dataClient.js`.
- You read Claude's explanation of why each is a real vulnerability.
- You fixed both and a second pass came back clean.

Lab 6 — Close the loop on DASH-147 with Cowork

~10 min · Claude Desktop · Prerequisites: Claude Desktop signed in; the `dash-147-security` folder with your fixes from Lab 5.

Goal. DASH-147 is fixed, reviewed, prototyped, and secured — but it has all stayed on your laptop, and Sarah, the PM who filed it, hasn't been told anything. Here you hand Cowork a *goal*: it reads the folder, writes the handoff itself, and updates the bug report in place.

What you'll learn

- The difference between **directing each step** (Code mode) and **handing over a goal** (Cowork) — when to stay in the loop for every edit, and when to let Claude work across several steps and check the finished result.
- How **permission and scope** keep that safe: Cowork only touches the folder you connect, and asks before it writes or deletes.
- That the "boring" last mile — the handoff, the ticket update, the status note — is real work Claude can take off your plate, grounded in what actually happened.

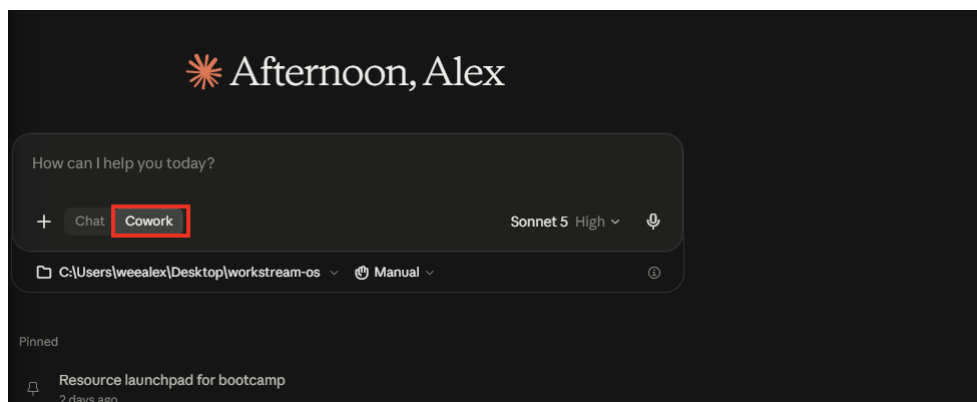
What Cowork does that Code mode doesn't

Code mode edits a codebase while you read along beside it. **Cowork takes a goal and works on its own** across several steps, then comes back with the result. You are not reviewing each edit as it happens — you approve the actions it wants to take and check the finished work. That is the whole point of this lab: everything so far needed you in the loop for each step, and a handoff document does not.

You approve every write. Cowork reads freely inside the folder you connect, but it asks before it writes or deletes anything. The folder you pick is the only part of your machine it can reach — nothing outside it is in scope.

Step 1 · Write the handoff document

- 1 In Claude Desktop, select **Cowork** (the mode selector in the message box, or the Cowork tab on some builds).
- 2 Choose **Select folder...** and pick the dash-147-security folder from Lab 5 — the one with your security fixes in it.
- 3 Leave the permission mode on the default, so Cowork asks before it writes anything.
- 4 Send the goal:



Claude Desktop home with Cowork selected in the message box, a folder connected, and permission mode set to Manual

Read the files in this folder, including bug-report.md, and check the git log to see what changed and when.

Then write a new file in this same folder called DASH-147-handoff.md. Base every statement on what you actually find in the files. Do not assume anything that is not there.

Include:

- What the bug was, in two sentences a non-engineer would understand
- The root cause, naming the specific files involved
- What changed to fix it
- What the security review found and what was done about it
- Anything still outstanding

- 1 Click **Allow** each time Cowork asks. When it finishes, open DASH-147-handoff.md in Finder/File Explorer and confirm it names the state-lifting root cause and the files that changed.

In real life. This is where Cowork earns its place: you give it an outcome — "read this folder and write the handoff" — and it does the multi-step work while you do something else. The same pattern turns a folder of PDFs into a comparison sheet, a quarter of notes into a briefing, or a project directory into a status summary. Notice the grounding instruction is the same one from Lab 1 — "base every statement on what you actually find" — because even when you hand over a goal, you still hold it to the evidence.

Watch out — if the file write fails. Some builds have trouble writing files. If that happens, ask Cowork for the same content as plain text in the conversation and save it yourself. The point of the lab is the read work write loop, not the file format.

Step 2 · Close the ticket in place

Now have Cowork update the original bug report rather than replacing it. Send this follow-up in the same conversation:

Now update bug-report.md in this folder, in place. Leave the original report text exactly as it is and add a Resolution section at the end that:

- States the bug is fixed and gives the root cause in one sentence
- Points to DASH-147-handoff.md for the detail
- Lists anything still outstanding that someone would need to pick up

- 1 Click **Allow** when Cowork asks to edit the file.
- 2 Open bug-report.md and confirm the Resolution section is at the end and the original report above it is untouched.
- 3 Read the outstanding list. It should name real follow-up work — for example that the Lab 5 security fixes are still uncommitted, or that the leaked API key was removed from source but never rotated. Cowork found those by reading the folder, not because you told it.

In real life. "Leave the original exactly as it is and add a Resolution section at the end" is a constraint you will reuse constantly — updating a document in place without letting Claude rewrite what was already agreed. And the fact that Cowork surfaced real outstanding work *you never mentioned* is the payoff of grounding: it reported what the folder actually showed, not what it assumed you wanted to hear.

Optional — put Cowork to work

- Ask Cowork to draft the three-sentence Slack message you'd send Sarah announcing the fix.
- Have it list every file it read to produce the handoff, and why each mattered.
- Ask for a short changelog entry in the style of the existing commit messages.
- Point Cowork at a folder from your own work and ask it to summarise what the project does.

You're done — the full loop, closed

- Cowork read the codebase and git history on its own.
- It wrote DASH-147-handoff.md, grounded in the actual files, naming the state-lifting root cause.
- It added a Resolution section to bug-report.md without disturbing the original.
- The outstanding list names real follow-up work Cowork found by itself.

Recap — what you actually practised

You carried one bug from a report to a closed ticket, and along the way you practised the habits that transfer to all of your own work:

- **Ground Claude in real context** — a Project with your files and instructions (Lab 1), Code mode pointed at the real repository (Lab 2), Cowork reading the actual folder (Lab 6). Generic answers come from missing context, not a weak model.
- **Explore and plan before you change anything** — diagnose the root cause first (Lab 1), tour the code and review the diff before it lands (Lab 2).
- **Give Claude a way to check its own work** — a reasoned code review (Lab 3), a security pass you re-run to confirm the fix (Lab 5), tracing the corrected flow back (Lab 2).
- **Match how much you supervise to the risk** — approve every edit on sensitive work, hand over a goal when the task is well-scoped and you can check the result (Labs 3 and 6).
- **Bring the work to the people who need it** — a clickable prototype for a stakeholder (Lab 4), a grounded handoff and a closed ticket for the team (Lab 6).

Take one of these back to a real task this week. The fastest way to build the intuition is to run the same loop — ground it, plan it, let it work, check it — on something that is actually yours.